

Ciencia de Datos para Economía y Negocios - FCE-UBA

Clase 5 — Visualización de datos con ggplot2

Nicolás Sidicaro

Hoja de ruta de hoy

- 1 Parte I: La lógica de ggplot2
- 2 Parte II: Construyendo un gráfico capa por capa
- 3 Parte III: Capas adicionales
- 4 Parte IV: El gráfico completo
- 5 Parte V: Mapa conceptual de ggplot2

Parte I: La lógica de ggplot2

¿Por qué ggplot2?

ggplot2 es el estándar de visualización en R para ciencia de datos.

Se basa en la **Grammar of Graphics** (Wilkinson, 1999; Wickham, 2010): un sistema formal para construir gráficos a partir de componentes.

Ventajas respecto al `plot()` de R base:

- Sintaxis **consistente y predecible**: siempre la misma lógica
- Gráficos **por capas**: cada elemento se agrega encima del anterior
- **Separación** entre datos, geometría y estética
- Altamente **personalizable** con el mismo vocabulario
- Miles de **extensiones** (animaciones, mapas, combinaciones de paneles...)

```
install.packages("ggplot2")    # o install.packages("tidyverse")  
library(ggplot2)               # o library(tidyverse)
```

Los datos que vamos a usar

Datos del **Banco Mundial** (WDI) para 2019: PBI per cápita (PPP, USD constantes de 2017) y esperanza de vida al nacer.

```
glimpse(wdi_2019)
```

	pais	anio	gdppc	lifeExp	region
1	Afghanistan	2019	2927.245	62.941	South Asia
2	Albania	2019	16761.193	79.467	Europe & Central Asia
3	Algeria	2019	15199.199	75.682	Middle East & North Africa
4	Andorra	2019	63215.900	84.098	Europe & Central Asia

La estructura mínima de un gráfico

Todo gráfico en ggplot2 tiene **tres componentes obligatorios**:

1. Los datos

Un dataframe (o tibble) con la información a visualizar.

2. Las estéticas (aes())

El mapeo entre variables del dataframe y propiedades visuales: eje x, eje y, color, tamaño, forma, etc.

3. La geometría (geom_*())

El tipo de gráfico: puntos, barras, líneas, cajas, histogramas...

```
ggplot(data = mis_datos, aes(x = variable_x, y = variable_y)) +  
  geom_point()
```

La metáfora de las capas

Pensemos en un gráfico como una **pila de acetatos** transparentes:

- 1 El **lienzo** base (`ggplot()`)
- 2 Una capa de **geometría** (`geom_*()`)
- 3 Ajustes de **escalas** (`scale_*()`)
- 4 **Etiquetas** y títulos (`labs()`)
- 5 Un **tema** visual (`theme_*()`, `theme()`)
- 6 **Facetas** para pequeños múltiples (`facet_*()`)

El operador +

En `ggplot2`, las capas se encadenan con **+**, **no** con el pipe `|>`. El pipe lleva los datos *hasta* `ggplot()`, y desde ahí usamos **+**.

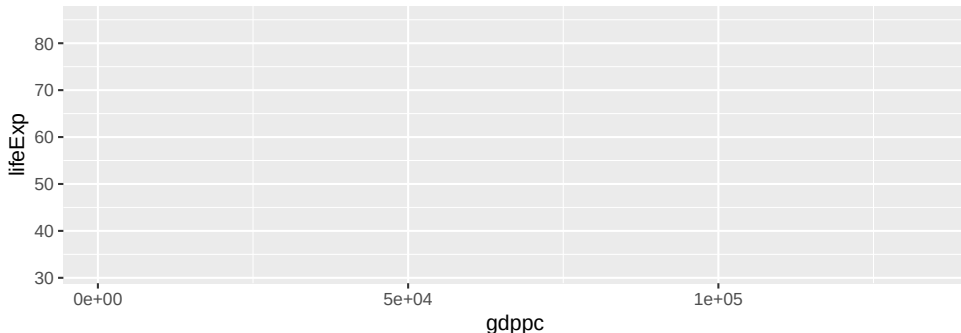
Cada capa es **opcional** (salvo los datos y la geometría). Podemos agregar o quitar capas sin romper el gráfico.

Parte II: Construyendo un gráfico capa por capa

Capa 0: el lienzo vacío

Empezamos con lo mínimo indispensable: **declarar los datos y las estéticas.**

```
ggplot(data = wdi_2019, aes(x = gdppc, y = lifeExp))
```

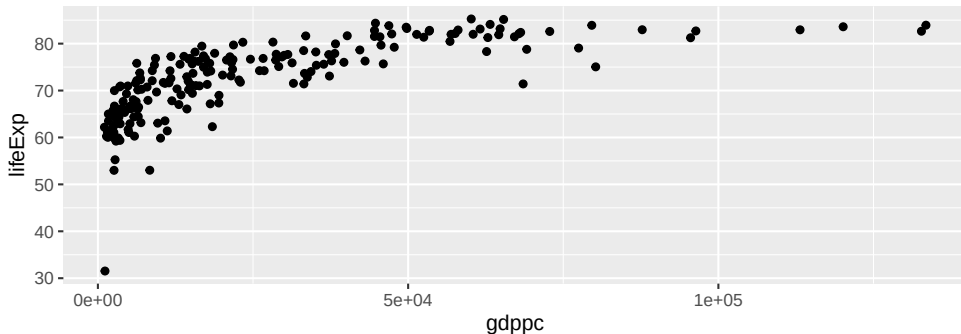


ggplot2 reconoce las variables y configura los ejes, pero no dibuja nada todavía porque no le dijimos **qué** dibujar.

Capa 1: agregar una geometría

Agregamos `geom_point()` para dibujar un punto por cada país.

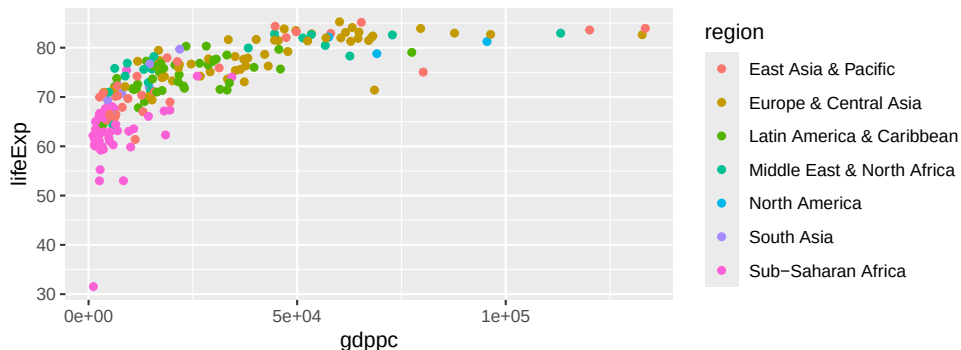
```
ggplot(data = wdi_2019, aes(x = gdppc, y = lifeExp)) +  
  geom_point()
```



Capa 1 (bis): otras estéticas dentro de aes()

Mapeamos una tercera variable al **color** de los puntos.

```
ggplot(data = wdi_2019, aes(x = gdppc, y = lifeExp, color = region)) +  
  geom_point()
```



El color aparece automáticamente y ggplot2 genera la leyenda.

¿Estética mapeada vs. estética fija?

Dentro de aes(): mapeada

El color/tamaño/forma **varía** según el valor de una variable.

- Genera una leyenda automática
- Refleja información del dato

```
aes(color = region) # variable
```

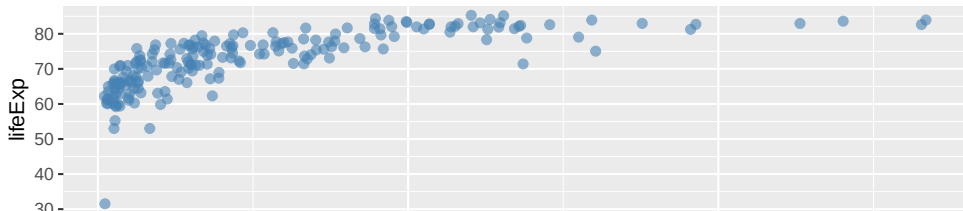
Fuera de aes(): fija

El color/tamaño/forma es **constante** para todos los puntos.

- No genera leyenda
- Es una decisión de diseño

```
geom_point(color = "steelblue")
```

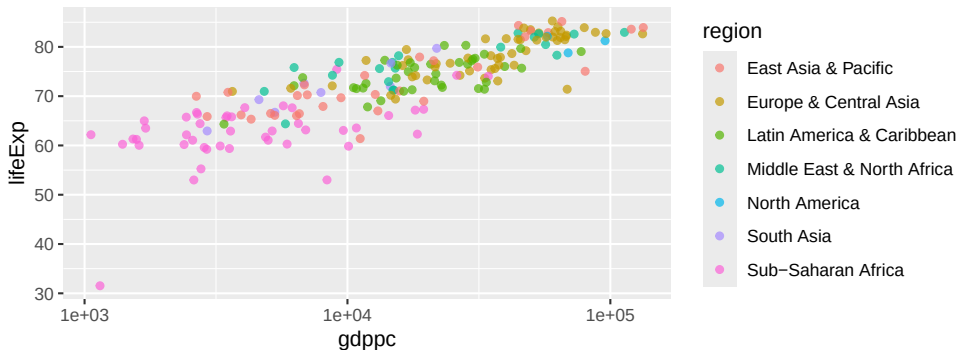
```
ggplot(data = wdi_2019, aes(x = gdppc, y = lifeExp)) +  
  geom_point(color = "steelblue", alpha = 0.6, size = 2)
```



Capa 2: ajustar escalas con `scale_*()`

El eje x tiene una distribución muy sesgada. Aplicamos **escala logarítmica**:

```
ggplot(data = wdi_2019, aes(x = gdppc, y = lifeExp, color = region)) +  
  geom_point(alpha = 0.7) +  
  scale_x_log10()
```



La familia `scale_*()`

Las funciones de escala siguen siempre el patrón: `scale_` + **estética** + `_` + **tipo**

Función	¿Qué modifica?
<code>scale_x_log10()</code> / <code>scale_y_log10()</code>	Escala logarítmica en los ejes
<code>scale_x_continuous()</code> / <code>scale_y_continuous()</code>	Eje numérico continuo (límites, breaks, etiquetas)
<code>scale_x_discrete()</code>	Eje con categorías
<code>scale_color_manual()</code>	Colores manuales para una variable discreta
<code>scale_color_gradient()</code>	Gradiente de color para variable continua
<code>scale_color_brewer()</code>	Paletas ColorBrewer (recomendadas)
<code>scale_size_continuous()</code>	Tamaño de puntos según una variable

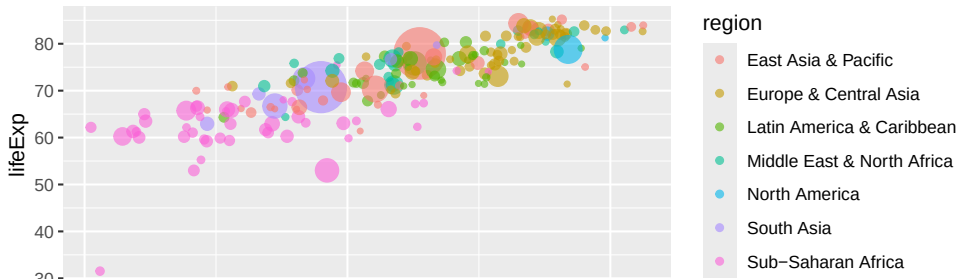
Regla

La estética que vas a modificar siempre aparece en el nombre de la función:
`scale_color_manual()`, `scale_x_log10()`, `scale_size_continuous()`.

Capa 2b: tamaño proporcional a una variable

range controla el tamaño mínimo y máximo de los puntos. guide = "none" oculta la leyenda de tamaño para no saturar el gráfico.

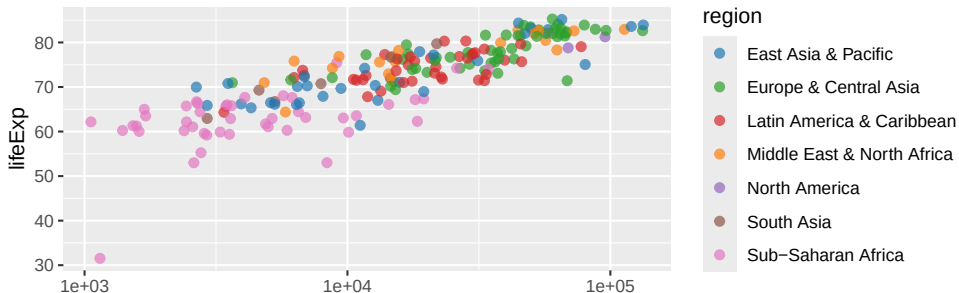
```
ggplot(data = wdi_2019,  
       aes(x = gdppc, y = lifeExp,  
           color = region, size = poblacion)) +  
  geom_point(alpha = 0.6) +  
  scale_x_log10() +  
  scale_size_continuous(range = c(1, 12), guide = "none")
```



Capa 2c: colores manuales con `scale_color_manual()`

En vez de dejar que ggplot2 elija, definimos **nuestra propia paleta**.

```
ggplot(data = wdi_2019,  
       aes(x = gdpppc, y = lifeExp, color = region)) +  
  geom_point(alpha = 0.7, size = 2) +  
  scale_x_log10() +  
  scale_color_manual(values = paleta_regiones)
```



Colores: ¿qué paleta usar?

Variables categóricas (sin orden)

Usar paletas **cualitativas**: colores distintos sin jerarquía.

- `scale_color_brewer(palette = "Set2")`
- `scale_color_manual()` con paleta propia

Variables continuas

Usar paletas **secuenciales** o **divergentes**.

- `scale_color_gradient(low, high)`
- `scale_color_gradient2()`
(divergente)
- `scale_color_viridis_c()`
(recomendada)

Reglas

- Nunca usar arcoíris (*rainbow*): engaña la percepción
- Usar siempre los mismos colores para las mismas categorías dentro de un documento
- Menos de 7 colores: el ojo empieza a confundirse

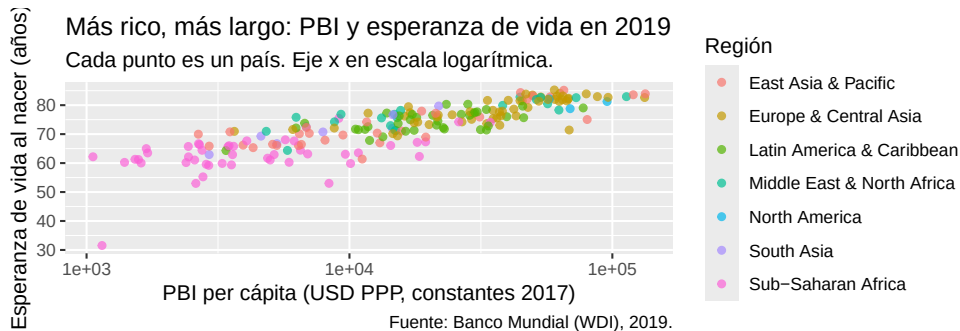
Capa 3: etiquetas con labs()

Agregamos título, subtítulo, etiquetas de ejes y fuente.

```
ggplot(data = wdi_2019, aes(x = gdppc, y = lifeExp, color = region)) +  
  geom_point(alpha = 0.7) +  
  scale_x_log10() +  
  labs(  
    title      = "Más rico, más largo: PBI y esperanza de vida en 2019",  
    subtitle   = "Cada punto es un país. Eje x en escala logarítmica.",  
    x          = "PBI per cápita (USD PPP, constantes 2017)",  
    y          = "Esperanza de vida al nacer (años)",  
    color      = "Región",  
    caption    = "Fuente: Banco Mundial (WDI), 2019."  
  )
```

Capa 3: etiquetas con labs()

Agregamos título, subtítulo, etiquetas de ejes y fuente.



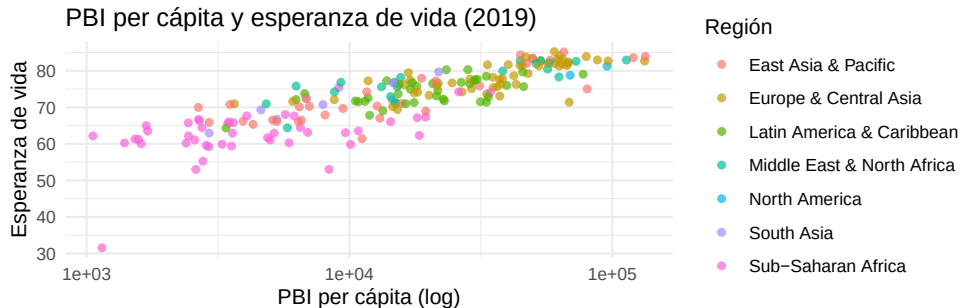
Capa 4: cambiar el tema con theme_*

Los temas controlan **todo lo que no es dato**: fondo, grillas, tipografía, márgenes.

```
ggplot(data = wdi_2019, aes(x = gdppc, y = lifeExp, color = region)) +  
  geom_point(alpha = 0.7) +  
  scale_x_log10() +  
  labs(title = "PBI per cápita y esperanza de vida (2019)",  
        x = "PBI per cápita (log)", y = "Esperanza de vida",  
        color = "Región", caption = "Fuente: WDI.") +  
  theme_minimal()
```

Capa 4: cambiar el tema con theme_*

Los temas controlan **todo lo que no es dato**: fondo, grillas, tipografía, márgenes.



Fuente: WDI.

Los temas predefinidos más usados

- `theme_gray()` — el tema por defecto (fondo gris)
- `theme_bw()` — fondo blanco, bordes negros
- `theme_minimal()` — minimalista, sin bordes
- `theme_classic()` — ejes clásicos, sin grilla
- `theme_light()` — grilla tenue, moderno
- `theme_void()` — sin ejes (útil para mapas)

Paquetes con más temas

- `ggthemes`: `theme_economist()`, `theme_fivethirtyeight()`
- `hrbrthemes`: tipografía limpia
- `ggpubr`: listos para publicación

¿Qué tema usar?

Temas por tema (?)

Para publicaciones académicas: `theme_minimal()` o `theme_bw()`. Para informes de política: `theme_light()` o temas personalizados. Evitar `theme_gray()` en presentaciones con proyector (el fondo gris oscurece).

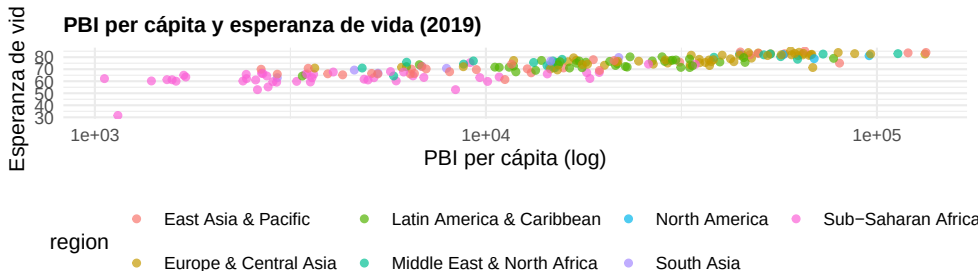
Capa 5: ajustes finos con theme()

theme() permite modificar **cualquier elemento individual** del gráfico.

```
ggplot(data = wdi_2019, aes(x = gdppc, y = lifeExp, color = region)) +  
  geom_point(alpha = 0.7) +  
  scale_x_log10() +  
  labs(title = "PBI per cápita y esperanza de vida (2019)",  
        x = "PBI per cápita (log)", y = "Esperanza de vida",  
        caption = "Fuente: WDI.") +  
  theme_minimal() +  
  theme(  
    legend.position = "bottom",          # leyenda abajo  
    plot.title      = element_text(face = "bold", size = 11),  
    plot.caption    = element_text(color = "gray50", hjust = 0)  
  )
```

Capa 5: ajustes finos con theme()

theme() permite modificar **cualquier elemento individual** del gráfico.



Fuente: WDI.

Elementos modificables con `theme()`

Argumento	¿Qué controla?	Función aplicable
<code>plot.title</code>	Título principal	<code>element_text()</code>
<code>plot.subtitle</code>	Subtítulo	<code>element_text()</code>
<code>plot.caption</code>	Fuente/nota al pie	<code>element_text()</code>
<code>axis.text</code>	Etiquetas de los ejes	<code>element_text()</code>
<code>axis.title</code>	Título de los ejes	<code>element_text()</code>
<code>legend.position</code>	Posición de la leyenda	"bottom", "top", "left", "none", <code>c(x, y)</code>
<code>legend.title</code>	Título de la leyenda	<code>element_text()</code> o <code>element_blank()</code>
<code>panel.grid</code>	Grilla de fondo	<code>element_line()</code> o <code>element_blank()</code>
<code>plot.background</code>	Fondo del gráfico completo	<code>element_rect()</code>
<code>panel.background</code>	Fondo del área de datos	<code>element_rect()</code>

Para borrar un elemento

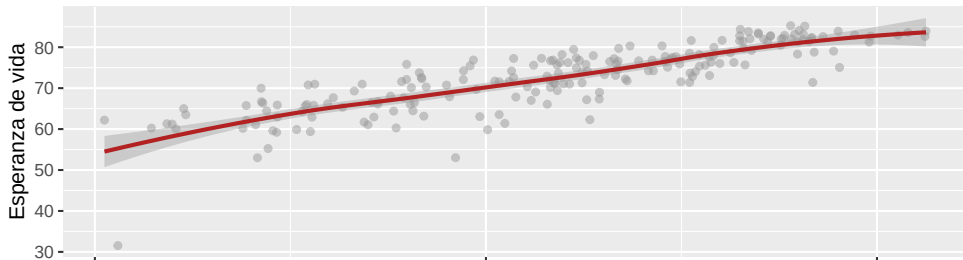
Usar `element_blank()`: elimina cualquier elemento visual. Ejemplo:
`theme(panel.grid.minor = element_blank())`.

Parte III: Capas adicionales

Capa 6: línea de tendencia con geom_smooth()

`geom_smooth()` ajusta y dibuja una curva encima de los puntos.

```
ggplot(data = wdi_2019, aes(x = gdppc, y = lifeExp)) +  
  geom_point(color = "gray60", alpha = 0.5) +  
  geom_smooth(method = "loess", color = "firebrick", se = TRUE) +  
  scale_x_log10() +  
  labs(x = "PBI per cápita (log)", y = "Esperanza de vida",  
       caption = "Fuente: WDI 2019.")
```



Opciones de geom_smooth()

Argumento	Efecto
method = "loess"	Curva suavizada local (defecto)
method = "lm"	Línea de regresión lineal
method = "glm"	Modelo lineal generalizado
se = TRUE	Muestra el intervalo de confianza
se = FALSE	Solo la línea, sin IC
color = "..."	Color de la línea
fill = "..."	Color del área del IC
linetype = ...	Tipo de línea (sólida, punteada...)

Importante

geom_smooth() usa **todos los datos**, incluso los que quedan fuera del área visible si aplicamos coord_cartesian() (lo vemos más adelante). Esto es una ventaja respecto a filtrar el dataframe.

```
# Regresión lineal simple
```

```
geom_smooth(method = "lm", se = FALSE, color = "navy")
```

Capa 7: etiquetar puntos con geom_text_repel()

`geom_text()` agrega texto, pero los rótulos se pisan. `ggrepel::geom_text_repel()` los **separa automáticamente**.

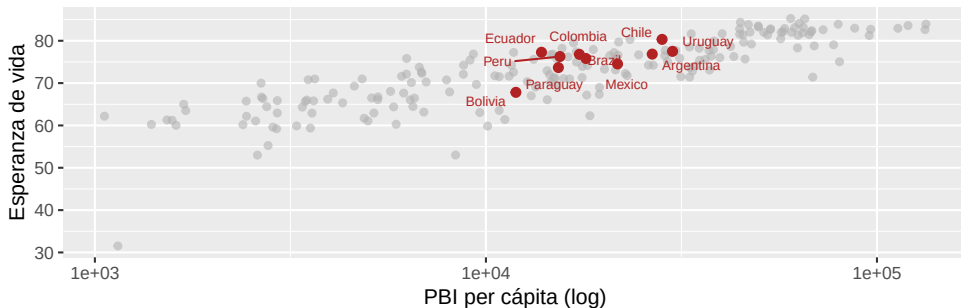
```
library(ggrepel)

ggplot(data = wdi_2019, aes(x = gdppc, y = lifeExp)) +
  geom_point(color = "gray70", alpha = 0.6) +
  geom_point(data = wdi_latam_2019, color = "firebrick", size = 2) +
  geom_text_repel(data = wdi_latam_2019, aes(label = pais),
                  size = 2.5, color = "firebrick",
                  max.overlaps = 15) +
  scale_x_log10() +
  labs(x = "PBI per cápita (log)", y = "Esperanza de vida",
       caption = "Fuente: WDI 2019.")
```



Capa 7: etiquetar puntos con geom_text_repel()

`geom_text()` agrega texto, pero los rótulos se pisan. `ggrepel::geom_text_repel()` los separa automáticamente.



Fuente: WDI 2019.

La lógica de los datos locales en una geometría

¿cómo resaltar algunos datos?

Datos globales vs. datos locales

Cada `geom_*()` puede recibir su **propio dataset** con el argumento `data =`. Esto sobrescribe el dataset global declarado en `ggplot()` *solo para esa capa*.

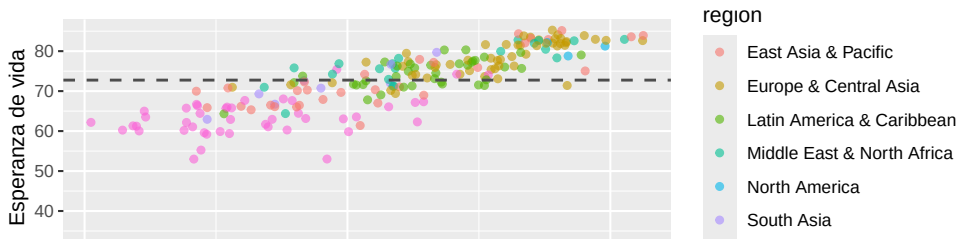
```
ggplot(data = wdi_2019, aes(x = gdppc, y = lifeExp)) +  
  geom_point(color = "gray70")           # usa wdi_2019 (global)  
  geom_point(data = wdi_latam_2019,      # usa solo los países de LATAM  
             color = "firebrick") +  
  geom_text_repel(data = wdi_latam_2019, # etiquetas solo para LATAM  
                  aes(label = pais))
```

Muy útil para **destacar un subgrupo** sin filtrar el dataset principal ni perder el contexto general.

Capa 8: líneas de referencia con geom_hline() / geom_vline()

Agregamos referencias que no vienen del dataset sino de **valores fijos** que queremos marcar.

```
ggplot(data = wdi_2019, aes(x = gdppc, y = lifeExp, color = region)) +  
  geom_point(alpha = 0.6) +  
  geom_hline(yintercept = promedio_mundial,  
             linetype = "dashed", color = "gray30", linewidth = 0.7) +  
  scale_x_log10() +  
  labs(x = "PBI per cápita (log)", y = "Esperanza de vida",  
       caption = "Fuente: WDI 2019. Línea punteada = promedio mundial.")
```



geom_hline(), geom_vline() y geom_abline()

Función	Argumento clave	Uso típico
geom_hline()	yintercept = valor	Referencia horizontal (promedio, umbral, cero)
geom_vline()	xintercept = valor	Referencia vertical (fecha, corte, mínimo legal)
geom_abline()	slope =, intercept =	Línea de 45° o recta de referencia teórica

Marcar el umbral de ingresos altos del Banco Mundial (~13,000 USD PPP)

```
geom_vline(xintercept = 13000,  
           linetype = "dotted", color = "navy", linewidth = 0.8)
```

Línea de 45° (por ejemplo, en gráficos de predicción vs. real)

```
geom_abline(slope = 1, intercept = 0, color = "gray50")
```

`geom_hline()`, `geom_vline()` y `geom_abline()`

Para recordar

Estas geometrías **no usan** `aes()`: reciben valores directamente. No mapean variables del dataframe, sino que dibujan en coordenadas fijas.

Capa 9: anotaciones con annotate()

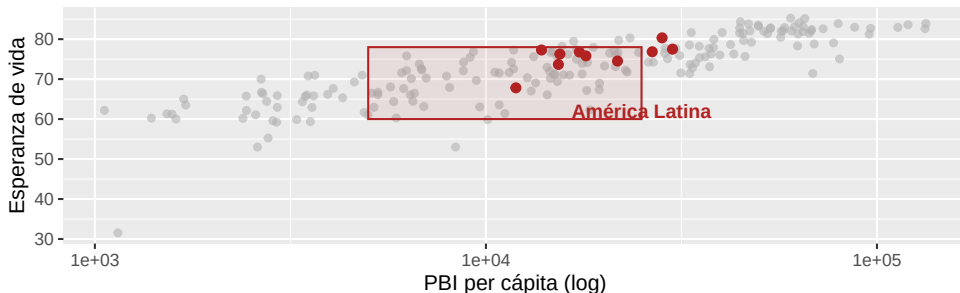
`annotate()` dibuja texto, flechas o formas en **coordenadas fijas** del gráfico. No usa columnas del dataset.

```
ggplot(data = wdi_2019, aes(x = gdppc, y = lifeExp)) +  
  geom_point(color = "gray70", alpha = 0.6) +  
  geom_point(data = wdi_latam_2019, color = "firebrick", size = 2) +  
  scale_x_log10() +  
  annotate("text", x = 25000, y = 62,  
          label = "América Latina", color = "firebrick",  
          fontface = "bold", size = 3.5) +  
  annotate("rect", xmin = 5000, xmax = 25000, ymin = 60, ymax = 78,  
          alpha = 0.08, fill = "firebrick", color = "firebrick") +  
  labs(x = "PBI per cápita (log)", y = "Esperanza de vida",  
       caption = "Fuente: WDI 2019.")
```



Capa 9: anotaciones con annotate()

`annotate()` dibuja texto, flechas o formas en **coordenadas fijas** del gráfico. No usa columnas del dataset.



Fuente: WDI 2019.

Tipos de anotaciones con annotate()

Texto en coordenadas fijas

```
annotate("text", x = 5000, y = 80,  
        label = "Mi etiqueta", size = 3, color = "navy")
```

Rectángulo de fondo (para destacar una zona)

```
annotate("rect", xmin = 1000, xmax = 10000, ymin = 70, ymax = 85,  
        alpha = 0.1, fill = "gold", color = "goldenrod")
```

Segmento / flecha

```
annotate("segment", x = 15000, xend = 50000, y = 76, yend = 83,  
        arrow = arrow(length = unit(0.2, "cm")), color = "gray40")
```

Área sombreada (banda horizontal)

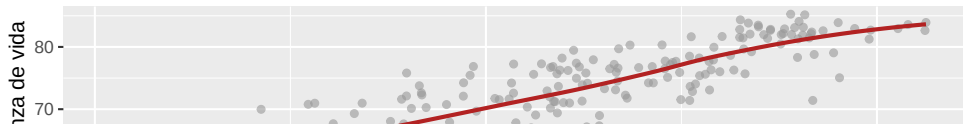
```
annotate("rect", xmin = -Inf, xmax = Inf, ymin = 75, ymax = 77,  
        alpha = 0.15, fill = "steelblue")
```

Capa 10: zoom con coord_cartesian()

`coord_cartesian()` hace zoom **sin eliminar datos**. Los puntos fuera del rango siguen existiendo para `geom_smooth()`.

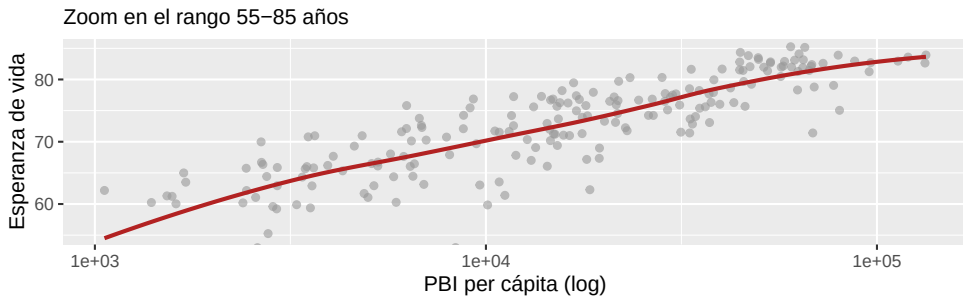
```
ggplot(data = wdi_2019, aes(x = gdppc, y = lifeExp)) +  
  geom_point(color = "gray60", alpha = 0.6) +  
  geom_smooth(method = "loess", color = "firebrick", se = FALSE) +  
  scale_x_log10() +  
  coord_cartesian(ylim = c(55, 85)) +  
  labs(x = "PBI per cápita (log)", y = "Esperanza de vida",  
       subtitle = "Zoom en el rango 55-85 años",  
       caption = "Fuente: WDI 2019.")
```

Zoom en el rango 55-85 años



Capa 10: zoom con coord_cartesian()

`coord_cartesian()` hace zoom **sin eliminar datos**. Los puntos fuera del rango siguen existiendo para `geom_smooth()`.



Fuente: WDI 2019.

coord_cartesian() vs. filtrar el dataset

Filtrar con filter()

Elimina las filas físicamente del dataset antes de graficar.

- `geom_smooth()` ignora los datos eliminados
- La curva cambia
- Útil cuando los outliers son errores de datos

```
wdi_2019 |>  
  filter(lifeExp > 55) |>  
  ggplot(...)
```

coord_cartesian()

Solo recorta la **ventana visual**. Los datos siguen ahí.

- `geom_smooth()` usa **todos** los datos
- La curva no cambia
- Útil para hacer zoom sin distorsionar el análisis

```
ggplot(...) + ... +  
  coord_cartesian(  
    ylim = c(55, 85))
```

También existe `scale_x_continuous(limits = ...)` pero **sí** descarta datos, como el filtro.

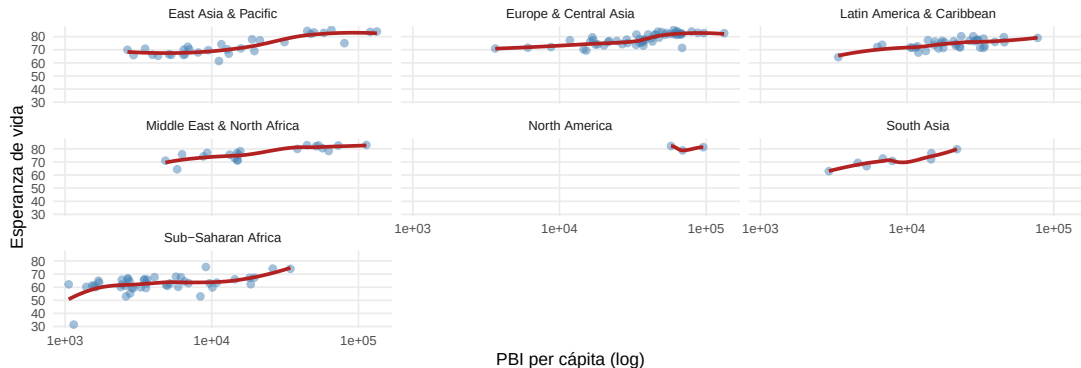
Capa 11: facetas con facet_wrap()

`facet_wrap()` divide el gráfico en **un panel por categoría**. Ideal para comparar la misma relación entre grupos.

```
ggplot(data = wdi_2019, aes(x = gdppc, y = lifeExp)) +  
  geom_point(alpha = 0.5, size = 1.2, color = "steelblue") +  
  geom_smooth(method = "loess", se = FALSE,  
              color = "firebrick", linewidth = 0.8) +  
  scale_x_log10() +  
  facet_wrap(~ region, ncol = 3) +  
  labs(x = "PBI per cápita (log)", y = "Esperanza de vida",  
       caption = "Fuente: WDI 2019.") +  
  theme_minimal(base_size = 8) +  
  theme(panel.grid.minor = element_blank())
```

Capa 11: facetas con facet_wrap()

`facet_wrap()` divide el gráfico en **un panel por categoría**. Ideal para comparar la misma relación entre grupos.



Fuente: WDI 2019.

facet_wrap() vs. facet_grid()

facet_wrap(~ var)

- Un panel por categoría de var
- Los paneles se acomodan en filas y columnas automáticamente
- `ncol = / nrow =` para controlar el layout
- `scales = "free"` para escalas independientes por panel

facet_grid(fila ~ col)

- Grilla cruzando **dos variables**
 - Útil cuando importa la posición relativa entre paneles
 - Ejemplo: región (filas) $\times$ nivel de ingreso (columnas)
- ```
facet_grid(income ~ region)
```

*# Escalas libres: cada panel tiene su propio rango*

```
facet_wrap(~ region, scales = "free")
```

*# Grilla de dos variables*

```
facet_grid(income ~ region)
```

## Parte IV: El gráfico completo

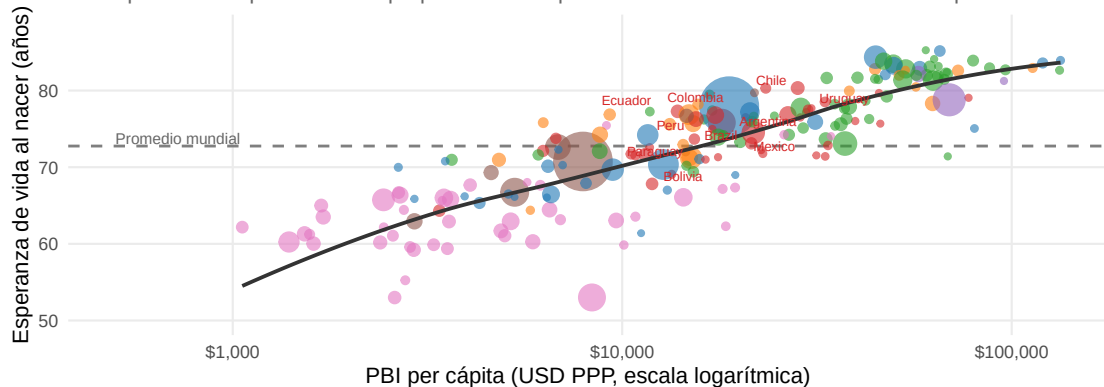
## El gráfico terminado — código completo

```
ggplot(data = wdi_2019,
 aes(x = gdpcc, y = lifeExp, color = region, size = poblacion)) +
 geom_hline(yintercept = promedio_mundial,
 linetype = "dashed", color = "gray50", linewidth = 0.6) +
 geom_point(alpha = 0.6) +
 geom_smooth(aes(group = 1), method = "loess",
 color = "gray20", se = FALSE, linewidth = 0.8,
 show.legend = FALSE) +
 geom_text_repel(data = wdi_latam_2019, aes(label = pais),
 size = 2.2, show.legend = FALSE, max.overlaps = 12) +
 annotate("text", x = 500, y = promedio_mundial + 1,
 label = "Promedio mundial", color = "gray40",
 size = 2.5, hjust = 0) +
 scale_x_log10(labels = scales::label_dollar(suffix = "")) +
 scale_size_continuous(range = c(1, 12), guide = "none") +
 scale_color_manual(values = paleta_regiones) +
 coord_cartesian(ylim = c(50, 88)) +
 labs(
 title = "Más rico, más vida: PBI per cápita y esperanza de vida",
```

# El gráfico terminado — resultado

## Más rico, más vida: PBI per cápita y esperanza de vida

Cada punto es un país. Tamaño proporcional a la población. Países de América Latina etiquetados. Año 20



### Región

- East Asia & Pacific
- Latin America & Caribbean
- North America
- Sub-Saharan Africa
- Europe & Central Asia
- Middle East & North Africa
- South Asia

# Lo que sumamos capa por capa

| #  | Capa             | Función                              | ¿Qué agrega?                     |
|----|------------------|--------------------------------------|----------------------------------|
| 0  | Lienzo           | <code>ggplot(data, aes())</code>     | Datos y ejes                     |
| 1  | Geometría        | <code>geom_point()</code>            | Los puntos                       |
| 1b | Más estéticas    | <code>aes(color=, size=)</code>      | Color y tamaño por variable      |
| 2  | Escala x         | <code>scale_x_log10()</code>         | Eje logarítmico                  |
| 2b | Escala tamaño    | <code>scale_size_continuous()</code> | Rango del tamaño                 |
| 2c | Escala color     | <code>scale_color_manual()</code>    | Paleta propia                    |
| 3  | Etiquetas        | <code>labs()</code>                  | Título, subtítulo, fuente, ejes  |
| 4  | Tema base        | <code>theme_minimal()</code>         | Estética general                 |
| 5  | Ajustes finos    | <code>theme()</code>                 | Leyenda abajo, título negrita... |
| 6  | Tendencia        | <code>geom_smooth()</code>           | Curva de ajuste LOESS            |
| 7  | Etiquetas puntos | <code>geom_text_repel()</code>       | Rótulos sin superposición        |
| 8  | Referencia       | <code>geom_hline()</code>            | Línea de promedio mundial        |
| 9  | Anotación        | <code>annotate()</code>              | Texto y formas en coord. fijas   |
| 10 | Zoom             | <code>coord_cartesian()</code>       | Recorte sin eliminar datos       |
| 11 | Facetas          | <code>facet_wrap()</code>            | Un panel por región              |

## Parte V: Mapa conceptual de ggplot2

# Las familias de funciones

| Familia       | Prefijo    | Ejemplos                                        |
|---------------|------------|-------------------------------------------------|
| Geometrías    | geom_*     | point, line, col, bar, smooth, hline, text, ... |
| Estadísticas  | stat_*     | smooth, summary, bin, ...                       |
| Escalas       | scale_*    | x_log10, color_manual, size_continuous, ...     |
| Coordenadas   | coord_*    | cartesian, flip, polar, map, fixed, ...         |
| Facetas       | facet_*    | wrap, grid                                      |
| Temas         | theme_*    | minimal, bw, classic, light, ...                |
| Ajustes finos | theme()    | Cualquier elemento visual individual            |
| Etiquetas     | labs()     | Todos los textos del gráfico                    |
| Anotaciones   | annotate() | Texto, flechas y formas en coord. fijas         |

## La lógica general

Cada familia modifica **una dimensión específica** del gráfico. Aprender ggplot2 es aprender qué familia usar para cada modificación que queremos hacer.

# Próxima parte: tipos de geometrías y colores

## Parte II de la clase (práctica)

- Un gráfico por tipo de geometría principal
  - `geom_col()` / `geom_bar()`: barras
  - `geom_line()`: series de tiempo
  - `geom_histogram()` / `geom_density()`: distribuciones
  - `geom_boxplot()`: cajas y bigotes
- Paletas de colores: `scale_color_brewer()`, `scale_fill_manual()` y cuándo usar qué paleta
- Ordenar barras con `fct_reorder()` (sin quedarnos con el orden alfabético)

## Bibliografía:

- Wickham, Çetinkaya-Rundel & Grolemund (2023). *R for Data Science*, 2da ed. Cap. 1 y 9–11. Disponible en: [r4ds.hadley.nz](https://r4ds.hadley.nz)
- Wickham (2016). *ggplot2: Elegant Graphics for Data Analysis*. [ggplot2-book.org](https://ggplot2-book.org)